Design e Arquitectura de Software

2022/2023

Lab. Classes - 2

GoF - Structural Patterns

1. Implement the basic **Proxy** pattern as illustrated in slide nr. 8.
   An example of expected output is:

```
The proxy call is happening now.
The doSomeWork() method is executed.
```

2. Implement a class `Adder` that implements a method that returns the summation of all the values in an array provided as argument in its constructor. Then, create a **virtual proxy** that caches the result so that the computation is not performed more than once.
   An example of expected output is:

```
CachingProxy: No cached result exists, calling original method
Adder: Computing sum. 1+2+3+4 = 10
The sum is 10
CachingProxy: A cached result exists, returning stored result
The sum is 10
```

3. Extend the previous exercise by creating a **protection proxy** that ensures that only authorized users can obtain the sum result. The proxy is initialized with a list of authorized users (strings). Only authorized users will be able to request the sum of the *Adder* object. To achieve this, the protection proxy method for obtaining the sum receives as argument the user id.

   An example of expected output is:

```
ProtectionProxy: User ANN is not authorized
Unauthorized Operation Exception thrown
ProtectionProxy: User JOHN is authorized
CachingProxy: No cached result exists, calling original method
Adder: Computing sum. 1+2+3+4 = 10
The sum is 10
ProtectionProxy: User PAUL is authorized
CachingProxy: A cached result exists, returning stored result
The sum is 10
```

4. Implement the **Decorator** pattern as illustrated in slides nr.14.

   Here is an example of expected output, with 6 different scenarios

```
Scenario 1:
Making a basic home with standard facilities.
The basic home with some standard facilities is ready.
You need to pay $100000.0 for this.
Total cost: $100000.0

Scenario 2:
Making a basic home. Then adding a swimming pool.
The basic home with some standard facilities is ready.
You need to pay $100000.0 for this.
For a swimming pool, you pay an extra $20000.0
Total cost: $120000.0

Scenario 3:

Making a basic home. Then adding two swimming pools one by one.
The basic home with some standard facilities is ready.
You need to pay $100000.0 for this.
For a swimming pool, you pay an extra $20000.0
For a swimming pool, you pay an extra $20000.0
Total cost: $140000.0

Scenario 4:
Making a basic home. Then adding one additional garden and
swimming pool.
The basic home with some standard facilities is ready.
You need to pay $100000.0 for this.
For a swimming pool, you pay an extra $20000.0
For a garden, you pay an extra $55000.0
Total cost: $175000.0

Scenario 5:
Making an advanced home.
The advanced home with more facilities is ready.
You need to pay $125000.0 for this.
Total cost: $125000.0

Scenario 6:
Making an advanced home. Then adding one additional
Swimming pool to it.
The advanced home with more facilities is ready
You need to pay $125000.0 for this.
For a swimming pool, you pay an extra $20000.0
Total cost: $145000.0
```

5. Implement the (object) **adapter** pattern as described in slide 19. Create a program that initializes and prints out a list of Figures, including Rectangles and Adapted Triangles.
   Here is an example of expected output

```
Processing the following objects:
Shape type: Rectangle.
Area: 200.0 square units.
Shape type: Triangle.
Area: 100.0 square units.
```

6. Change the previous exercise so that a *class* **adapter** is used instead.

7. Implement the **Facade** pattern as illustrated in slide 23.
   Here is an example of expected output

```
Checking the loan approval status of Bob
[The current asset value: 5000.0,
claim amount: 20000.0,
existing loan?: true ]
Verifying Bob's asset value.
Verifying Bob's previous loan(s) status.
Bob's application status: Not approved.
Remarks if any:
Insufficient balance.
An old loan exists.
--------
Checking the loan approval status of Jack
[The current asset value: 70000.0,
claim amount: 30000.0,
existing loan?: false ]
Verifying Jack's asset value.
Verifying Jack's previous loan(s) status.
Jack's application status: Approved
Remarks if any:
--------
Checking the loan approval status of Tony
[The current asset value: 125000.0,
claim amount: 125000.0,
existing loan?: true ]
Verifying Tony's asset value.
Verifying Tony's previous loan(s) status.
Tony's application status: Not approved.
Remarks if any:
An old loan exists.
--------
```

8. Implement the **Flyweight** pattern, as described in slide 30. Consider brand and model as instrinc, and owner and color as extrinsic.
   Here is an example of expected output

```
Making a car Audi A4 for the first time.
John owns a green Audi A4
Reusing an existing car
Marc owns a blue Audi A4
Making a car Subaru Rex for the first time.
Marc owns a pink Subaru Rex

Number of car objects used in this example: 2
```

9. Implement the **Composite** pattern, as described in slide 35.
   Here is an example of expected output

```
Creating a Picture (picture 1)
Creating a Picture (picture 2)
Creating a text (text 1)
Adding text 1 to picture 2
Creating a line (line 1)
Adding line 1 to picture 2
Adding picture 2 to picture 1
Creating a line (line 2)
Adding line 2 to picture 1
Creating a rectangle (rectangle 1)
Adding rectangle 1 to picture 1
--------
Started drawing composite picture 1
Started drawing composite picture 2
Drawing text 1
Drawing line 1
Finished drawing composite picture 2
Drawing line 2
Drawing rectangle 1
finished drawing composite picture 1.
```

10. Implement the **Bridge** pattern, as described in slide 42.
    Here is an example of expected output

```
Creating a new IconWindow (icon window 1)
Setting implementation to XWindowImpl (xwindowimpl)in icon
window 1
Drawing border in icon window 1
Drawing rect in icon window 1
devDrawLine in xwindowimpl
devDrawLine in xwindowimpl
devDrawLine in xwindowimpl
devDrawLine in xwindowimpl
Drawing text in icon window 1
devDrawText in xwindowimpl

Setting implementation to PMWindowImpl (pmwindowimpl)in icon
window 1
Drawing border in icon window 1
Drawing rect in icon window 1
devDrawLine in pmwindowimpl
devDrawLine in pmwindowimpl
devDrawLine in pmwindowimpl
devDrawLine in pmwindowimpl
Drawing text in icon window 1
devDrawText in pmwindowimpl
```